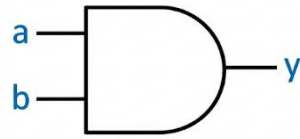


AND Gate

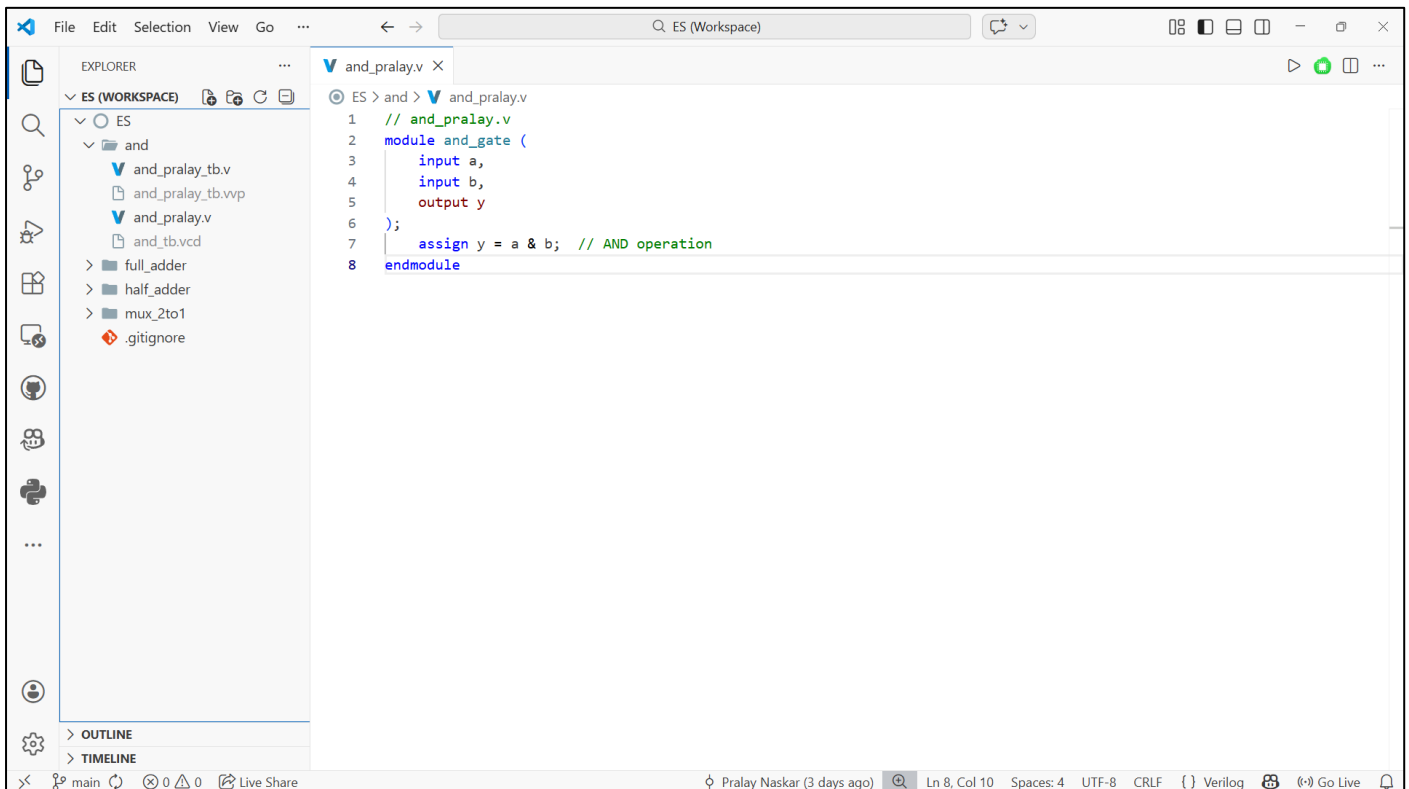
AND Gate: Symbol and Truth Table



Input a	Input b	Output y
0	0	0
0	1	0
1	0	0
1	1	1

Verilog Code

```
// and_pralay.v
module and_gate (
    input a,
    input b,
    output y
);
    assign y = a & b; // AND operation
endmodule
```



```

// and_pralay_tb.v
`timescale 1ns/1ps
module and_gate_tb;

    // Testbench signals
    reg a;
    reg b;
    wire y;

    // Instantiate the AND gate
    and_gate uut (
        .a(a),
        .b(b),
        .y(y)
    );

    // Stimulus
    initial begin
        $dumpfile("and_tb.vcd");
        $dumpvars(0, and_gate_tb);

        $display("A B | Y");
        $display("-----");

        a = 0; b = 0; #10;
        $display("%b %b | %b", a, b, y);

        a = 0; b = 1; #10;
        $display("%b %b | %b", a, b, y);

        a = 1; b = 0; #10;
        $display("%b %b | %b", a, b, y);

        a = 1; b = 1; #10;
        $display("%b %b | %b", a, b, y);

        $finish;
    end

endmodule

```

The screenshot shows the VS Code editor with a workspace named 'ES (Workspace)'. The Explorer panel on the left shows a project structure with files like 'and', 'and_pralay_tb.v', 'and_pralay_tb.vvp', 'and_pralay.v', 'and_tb.vcd', 'full_adder', 'half_adder', 'mux_2to1', and '.gitignore'. The main editor window displays the Verilog code for 'and_pralay_tb.v'.

```
1 | // and_pralay_tb.v
2 | `timescale 1ns/1ps
3 | module and_gate_tb;
4 |
5 |     // Testbench signals
6 |     reg a;
7 |     reg b;
8 |     wire y;
9 |
10 |    // Instantiate the AND gate
11 |    and_gate uut (
12 |        .a(a),
13 |        .b(b),
14 |        .y(y)
15 |    );
16 |
17 |    // Stimulus
18 |    initial begin
19 |        $dumpfile("and_tb.vcd");
20 |        $dumpvars(0, and_gate_tb);
21 |
22 |        $display("A B | Y");
23 |        $display("-----");
24 |
25 |        a = 0; b = 0; #10;
26 |        $display("%b %b | %b", a, b, y);
27 |
28 |        a = 0; b = 1; #10;
29 |        $display("%b %b | %b", a, b, y);
30 |
31 |        a = 1; b = 0; #10;
32 |        $display("%b %b | %b", a, b, y);
33 |
34 |        a = 1; b = 1; #10;
35 |        $display("%b %b | %b", a, b, y);
36 |
37 |        $finish;
38 |    end
39 |
40 | endmodule
```

OUTPUT

The screenshot shows the VS Code editor with the terminal window open, displaying the output of the simulation. The terminal output shows the command to run the simulation and the resulting waveforms. A GTKWave window is also open, showing the simulation results for the AND gate testbench.

Terminal Output:

```
PS C:\Users\PRALAY\Documents\Test\ES> cd and
PS C:\Users\PRALAY\Documents\Test\ES\and> iverilog -o and_pralay_tb.vvp and_pralay.v and_pralay_tb.v
PS C:\Users\PRALAY\Documents\Test\ES\and> vvp and_pralay_tb.vvp
VCD info: dumpfile and_tb.vcd opened for output.
A B | Y
-----
0 0 | 0
0 1 | 0
1 0 | 0
1 1 | 1
and_pralay_tb.v:37: $finish called at 40000 (1ps)
PS C:\Users\PRALAY\Documents\Test\ES\and> gtkwave and_tb.vcd

GTKWave Analyzer v3.3.100 (w)1999-2019 BSI

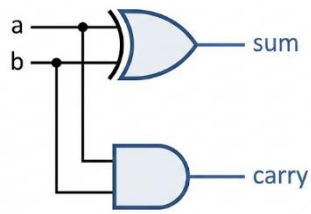
[0] start time.
[40000] end time.
```

GTKWave Window:

The GTKWave window shows the simulation results for the AND gate testbench. The Signals panel on the left lists the signals: 'and_gate_tb', 'a', 'b', and 'y'. The Waveform panel on the right displays the timing diagram for these signals. The time scale is set to 10 ns, and the cursor is at 18260 ps. The waveform shows that the output 'y' is 0 for the first three input combinations (00, 01, 10) and 1 for the fourth combination (11).

Half-Adder Gate

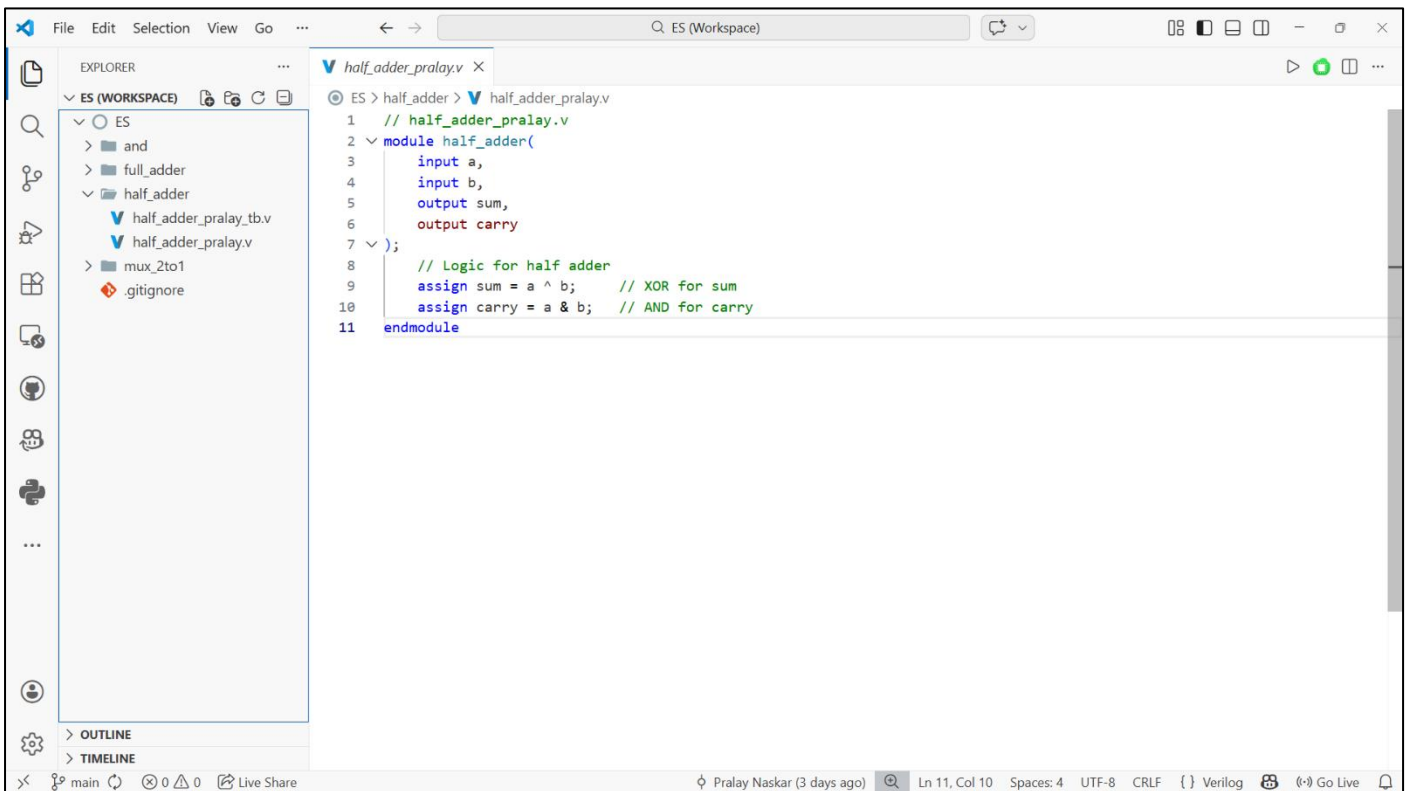
Half-Adder Circuit and Truth Table



Input		Output	
a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Verilog Code

```
// half_adder_pralay.v
module half_adder(
    input a,
    input b,
    output sum,
    output carry
);
    // Logic for half adder
    assign sum = a ^ b;    // XOR for sum
    assign carry = a & b;  // AND for carry
endmodule
```



```

// half_adder_pralay_tb.v
`timescale 1ns/1ps
module half_adder_tb;

    reg a;
    reg b;
    wire sum;
    wire carry;

    // Instantiate the half adder
    half_adder uut (
        .a(a),
        .b(b),
        .sum(sum),
        .carry(carry)
    );

    // Test sequence
    initial begin
        $dumpfile("half_adder.vcd");    // VCD file for GTKWave
        $dumpvars(0, half_adder_tb);

        $display("A B | Sum Carry");
        $display("-----");

        a = 0; b = 0; #10;
        $display("%b %b | %b %b", a, b, sum, carry);

        a = 0; b = 1; #10;
        $display("%b %b | %b %b", a, b, sum, carry);

        a = 1; b = 0; #10;
        $display("%b %b | %b %b", a, b, sum, carry);

        a = 1; b = 1; #10;
        $display("%b %b | %b %b", a, b, sum, carry);

        $finish;
    end

endmodule

```

The screenshot shows the VS Code editor with a Verilog testbench file named `half_adder_pralay_tb.v`. The code defines a module `half_adder_tb` that instantiates a `half_adder` module and includes a test sequence. The test sequence sets initial values for `a` and `b` and checks the output `sum` and `carry` for various input combinations.

```
1 // half_adder_pralay_tb.v
2 `timescale 1ns/1ps
3 module half_adder_tb;
4
5     reg a;
6     reg b;
7     wire sum;
8     wire carry;
9
10    // Instantiate the half adder
11    half_adder uut (
12        .a(a),
13        .b(b),
14        .sum(sum),
15        .carry(carry)
16    );
17
18    // Test sequence
19    initial begin
20        $dumpfile("half_adder.vcd"); // VCD file for GTKWave
21        $dumpvars(0, half_adder_tb);
22
23        $display("A B | Sum Carry");
24        $display("-----");
25
26        a = 0; b = 0; #10;
27        $display("%b %b | %b %b", a, b, sum, carry);
28
29        a = 0; b = 1; #10;
30        $display("%b %b | %b %b", a, b, sum, carry);
31
32        a = 1; b = 0; #10;
33        $display("%b %b | %b %b", a, b, sum, carry);
34
35        a = 1; b = 1; #10;
36        $display("%b %b | %b %b", a, b, sum, carry);
37
38        $finish;
39    end
40
41 endmodule
```

OUTPUT

The screenshot shows the VS Code editor with the terminal output of the simulation. The output includes the command to run the simulation and the resulting waveforms. A GTKWave window is also open, displaying the waveforms for the signals `a`, `b`, `carry`, and `sum`.

Terminal Output:

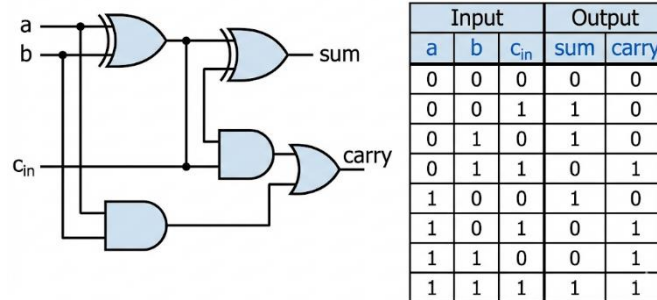
```
PS C:\Users\PRALAY\Documents\Test\ES> cd half_adder
PS C:\Users\PRALAY\Documents\Test\ES\half_adder> verilog -o half_adder_pralay_tb.vvp half_adder_pralay.v half_adder_pralay_tb.v
PS C:\Users\PRALAY\Documents\Test\ES\half_adder> vvp half_adder_pralay_tb.vvp
VCD info: dumpfile half_adder.vcd opened for output.
A B | Sum Carry
0 0 | 0 0
0 1 | 1 0
1 0 | 1 0
1 1 | 0 1
half_adder_pralay_tb.v:38: $finish called at 40000 (1ps)
PS C:\Users\PRALAY\Documents\Test\ES\half_adder> gtkwave half_adder.vcd
GTKWave Analyzer v3.3.100 (w)1999-2019 BSI
[0] start time.
[40000] end time.
```

GTKWave - half_adder.vcd

The GTKWave window shows the simulation results for the signals `a`, `b`, `carry`, and `sum`. The waveforms are displayed over a time range from 0 to 40 ns. The signals `a` and `b` are square waves, and the signals `carry` and `sum` are also square waves, showing the output of the half-adder circuit.

Full-Adder

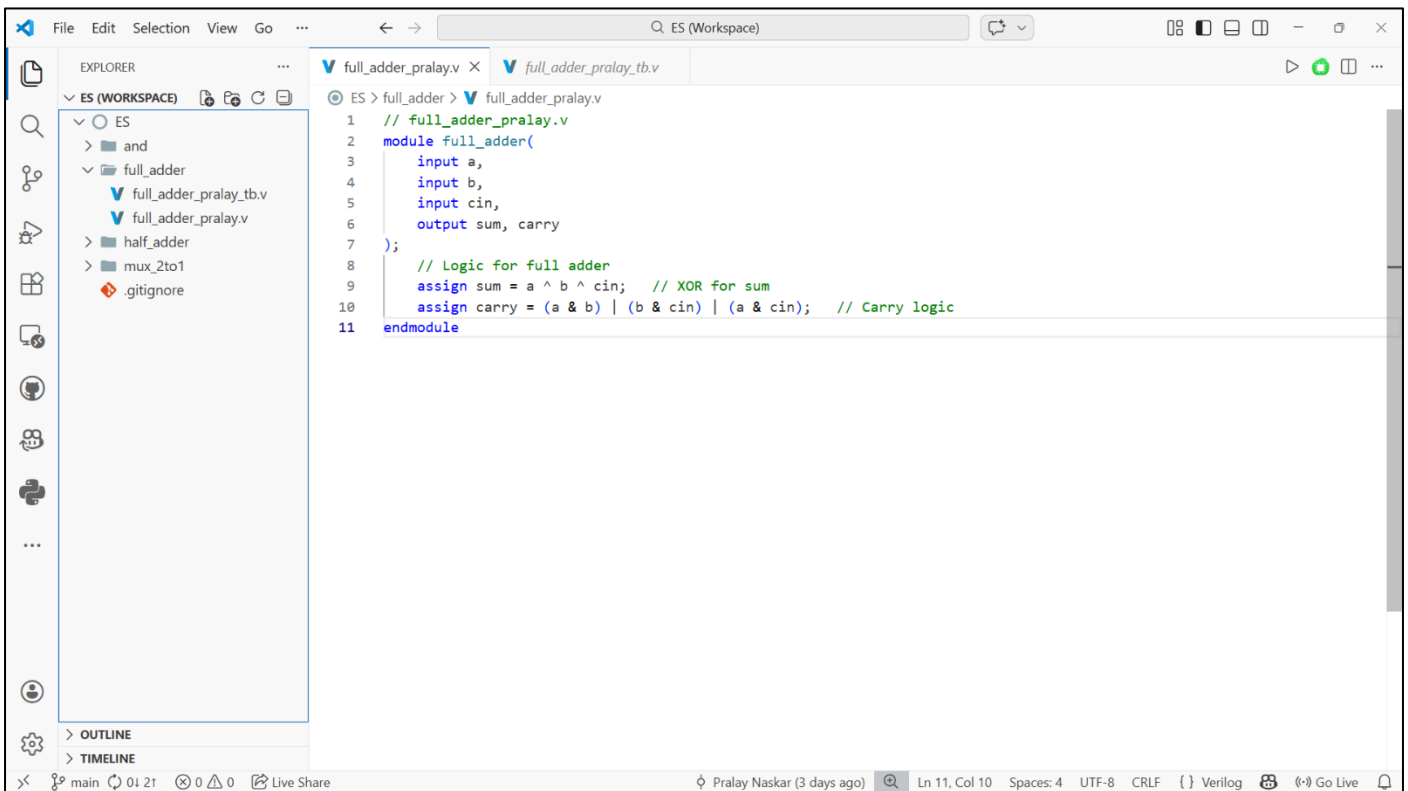
Full-Adder Circuit and Truth Table



Verilog Code

```
// full_adder_pralay.v
module full_adder(
    input a,
    input b,
    input cin,
    output sum, carry
);

    // Logic for full adder
    assign sum = a ^ b ^ cin;    // XOR for sum
    assign carry = (a & b) | (b & cin) | (a & cin);    // Carry Logic
endmodule
```



```

// full_adder_pralay_tb.v
`timescale 1ns/1ps
module full_adder_tb;

    reg a;
    reg b;
    reg cin;
    wire sum;
    wire carry;

    // Instantiate the full adder
    full_adder uut (
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum),
        .carry(carry)
    );

    // Test sequence
    initial begin
        $dumpfile("full_adder.vcd"); // VCD file for GTKWave
        $dumpvars(0, full_adder_tb);

        $display("A B Cin | Sum Carry");
        $display("-----");

        a = 0; b = 0; cin = 0; #10;
        $display("%b %b %b | %b %b", a, b, cin, sum, carry);

        a = 0; b = 0; cin = 1; #10;
        $display("%b %b %b | %b %b", a, b, cin, sum, carry);

        a = 0; b = 1; cin = 0; #10;
        $display("%b %b %b | %b %b", a, b, cin, sum, carry);

        a = 0; b = 1; cin = 1; #10;
        $display("%b %b %b | %b %b", a, b, cin, sum, carry);

        a = 1; b = 0; cin = 0; #10;
        $display("%b %b %b | %b %b", a, b, cin, sum, carry);

        a = 1; b = 0; cin = 1; #10;
        $display("%b %b %b | %b %b", a, b, cin, sum, carry);

        a = 1; b = 1; cin = 0; #10;
        $display("%b %b %b | %b %b", a, b, cin, sum, carry);

        a = 1; b = 1; cin = 1; #10;
        $display("%b %b %b | %b %b", a, b, cin, sum, carry);

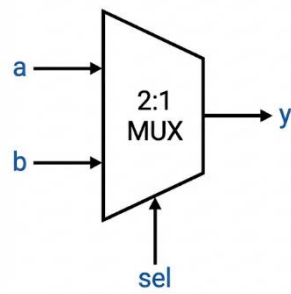
        $finish;
    end

endmodule

```


2 Input Multiplexer

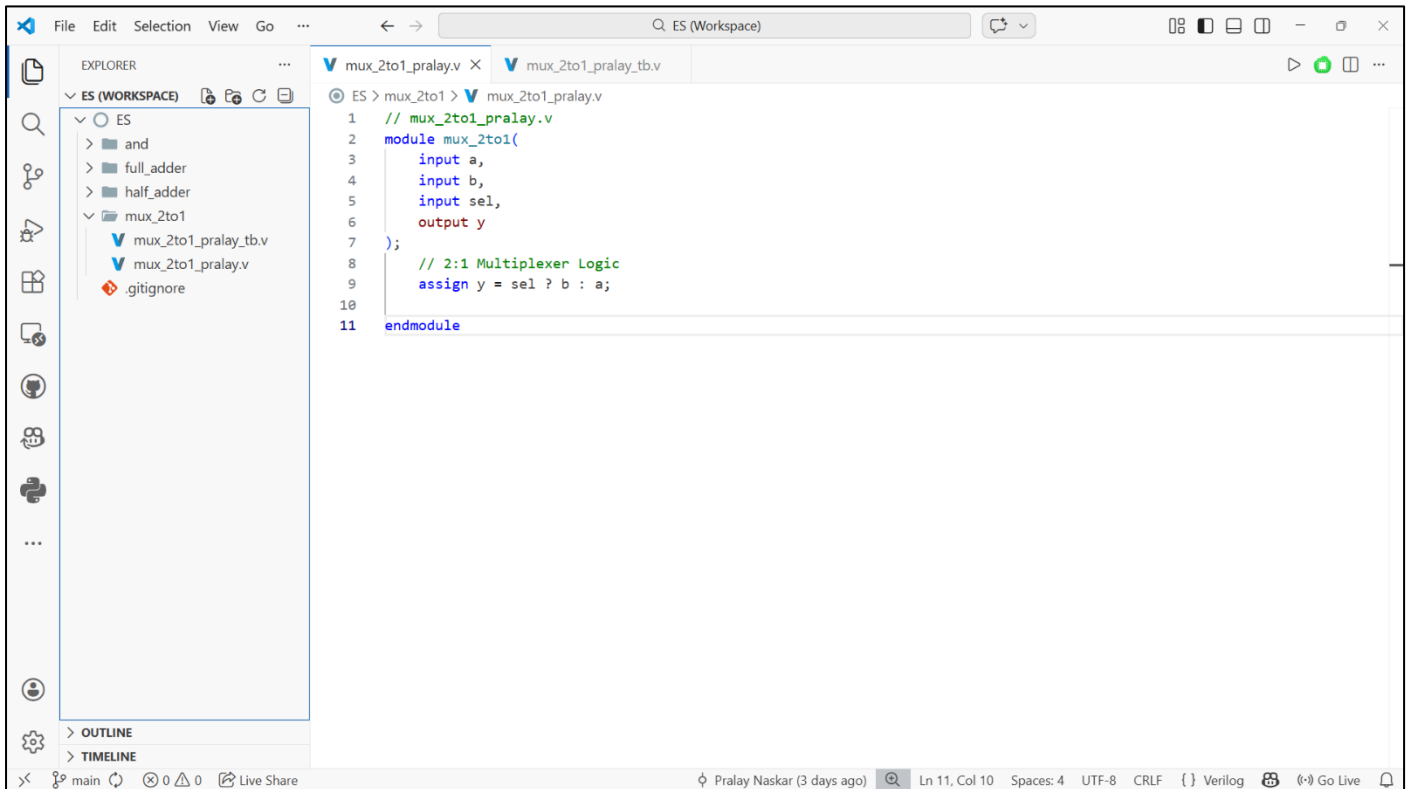
2-Input Multiplexer (MUX) Circuit and Truth Table



Input sel	Input a	Input b	Output y
0	X	X	a
1	X	X	b
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Verilog Code

```
// mux_2to1_pralay.v
module mux_2to1(
    input a,
    input b,
    input sel,
    output y
);
    // 2:1 Multiplexer Logic
    assign y = sel ? b : a;
endmodule
```



```

// mux_2to1_pralay_tb.v
`timescale 1ns/1ps
module mux_2to1_tb;

    reg a;
    reg b;
    reg sel;
    wire y;

    // Instantiate the 2:1 MUX
    mux_2to1 uut (
        .a(a),
        .b(b),
        .sel(sel),
        .y(y)
    );

    // Test sequence
    initial begin
        $dumpfile("mux_2to1.vcd"); // VCD file for GTKWave
        $dumpvars(0, mux_2to1_tb);

        $display("A B Sel | Y");
        $display("-----");

        a = 0; b = 0; sel = 0; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        a = 0; b = 1; sel = 0; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        a = 1; b = 0; sel = 0; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        a = 1; b = 1; sel = 0; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        a = 0; b = 0; sel = 1; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        a = 0; b = 1; sel = 1; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        a = 1; b = 0; sel = 1; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        a = 1; b = 1; sel = 1; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        $finish;
    end
endmodule

```

```
1 // mux_2to1_tb.v
2 `timescale 1ns/1ps
3 module mux_2to1_tb;
4
5     reg a;
6     reg b;
7     reg sel;
8     wire y;
9
10    // Instantiate the 2:1 MUX
11    mux_2to1 u1 (
12        .a(a),
13        .b(b),
14        .sel(sel),
15        .y(y)
16    );
17
18    // Test sequence for all possible input combinations
19    initial begin
20
21        $dumpfile("mux_2to1.vcd");
22        $dumpvars(0, mux_2to1_tb);
23
24        $display("A B Sel | Y");
25        $display("-----");
26
27        a = 0; b = 0; sel = 0; #10;
28        $display("%0 Nd %0 Nd | %0", a, b, sel, y);
29
30        a = 0; b = 1; sel = 0; #10;
31        $display("%0 Nd %0 Nd | %0", a, b, sel, y);
32
33        a = 1; b = 0; sel = 0; #10;
34        $display("%0 Nd %0 Nd | %0", a, b, sel, y);
35
36        a = 1; b = 1; sel = 0; #10;
37        $display("%0 Nd %0 Nd | %0", a, b, sel, y);
38
39        a = 0; b = 0; sel = 1; #10;
40        $display("%0 Nd %0 Nd | %0", a, b, sel, y);
41
42        a = 0; b = 1; sel = 1; #10;
43        $display("%0 Nd %0 Nd | %0", a, b, sel, y);
44
45        a = 1; b = 0; sel = 1; #10;
46        $display("%0 Nd %0 Nd | %0", a, b, sel, y);
47
48        a = 1; b = 1; sel = 1; #10;
49        $display("%0 Nd %0 Nd | %0", a, b, sel, y);
50
51        $finish;
52    end
53 endmodule
```

OUTPUT

PS C:\Users\PRALAY\Documents\Test\ES> cd mux_2to1

PS C:\Users\PRALAY\Documents\Test\ES\mux_2to1> iverilog -o mux_2to1_pralay_tb.vvp mux_2to1_pralay.v mux_2to1_pralay_tb.v

PS C:\Users\PRALAY\Documents\Test\ES\mux_2to1> vvp mux_2to1_pralay_tb.vvp

VCD info: dumpfile mux_2to1.vcd opened for output.

A B Sel | Y

0 0 0 | 0

0 1 0 | 0

1 0 0 | 1

1 1 0 | 1

0 0 1 | 0

0 1 1 | 1

1 0 1 | 0

1 1 1 | 1

mux_2to1_pralay_tb.v:51: \$finish called at 80000 (1ps)

PS C:\Users\PRALAY\Documents\Test\ES\mux_2to1> gtkwave mux_2to1.vcd

GTKWave Analyzer v3.3.100 (w)1999-2019 BSI

[0] start time.

[80000] end time.

GTKWave - mux_2to1.vcd

File Edit Search Time Markers View Help

From: 0 sec To: 80 ns Marker: -- Cursor: 47300 ps

Signals:

- mux_2to1_tb
- u1

Waves:

- a
- b
- sel
- y

Type Signals

wire a

wire b

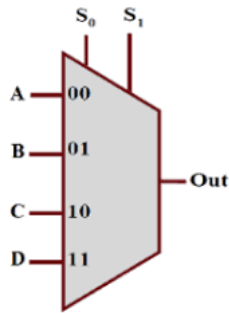
wire sel

wire y

Filter:

Append Insert Replace

4 Input Multiplexer implementation on a FPGA board



S ₁	S ₀	A	B	C	D	Out
0	0	0	x	x	x	0
0	0	1	x	x	x	1
0	1	x	0	x	x	0
0	1	x	1	x	x	1
1	0	x	x	0	x	0
1	0	x	x	1	x	1
1	1	x	x	x	0	0
1	1	x	x	x	1	1

Verilog Code

```
// mux_4to1_pralay.v
module mux_4to1(
    input a, b, c, d,
    input s0, s1,
    output y
);
    // 4:1 Multiplexer Logic
    assign y = (s1) ? ((s0) ? d : c) : ((s0) ? b : a);
endmodule
```

```
// mux_4to1_pralay_tb.v
`timescale 1ns/1ps
module mux_4to1_tb;

    // Inputs
    reg a, b;
    reg c, d;
    reg s0, s1;

    // Output
    wire y;

    // Instantiate the Unit Under Test (UUT)
    mux_4to1 uut (
        .a(a), .b(b),
        .c(c), .d(d),
        .s0(s0), .s1(s1),
        .y(y)
    );

    initial begin
        $dumpfile("mux_4to1.vcd");
        $dumpvars(0, mux_4to1_tb);

        $display("A B C D S1 S0 | Y");
        $display("-----");
    end
endmodule
```

```

// Test Case 1: Select A (s1=0, s0=0)
a = 1; b = 0; c = 0; d = 0; s1 = 0; s0 = 0; #10;
$display("%b %b %b %b %b %b | %b", a, b, c, d, s1, s0,
y);

// Test Case 2: Select B (s1=0, s0=1)
a = 0; b = 1; c = 0; d = 0; s1 = 0; s0 = 1; #10;
$display("%b %b %b %b %b %b | %b", a, b, c, d, s1, s0,
y);

// Test Case 3: Select C (s1=1, s0=0)
a = 0; b = 0; c = 1; d = 0; s1 = 1; s0 = 0; #10;
$display("%b %b %b %b %b %b | %b", a, b, c, d, s1, s0,
y);

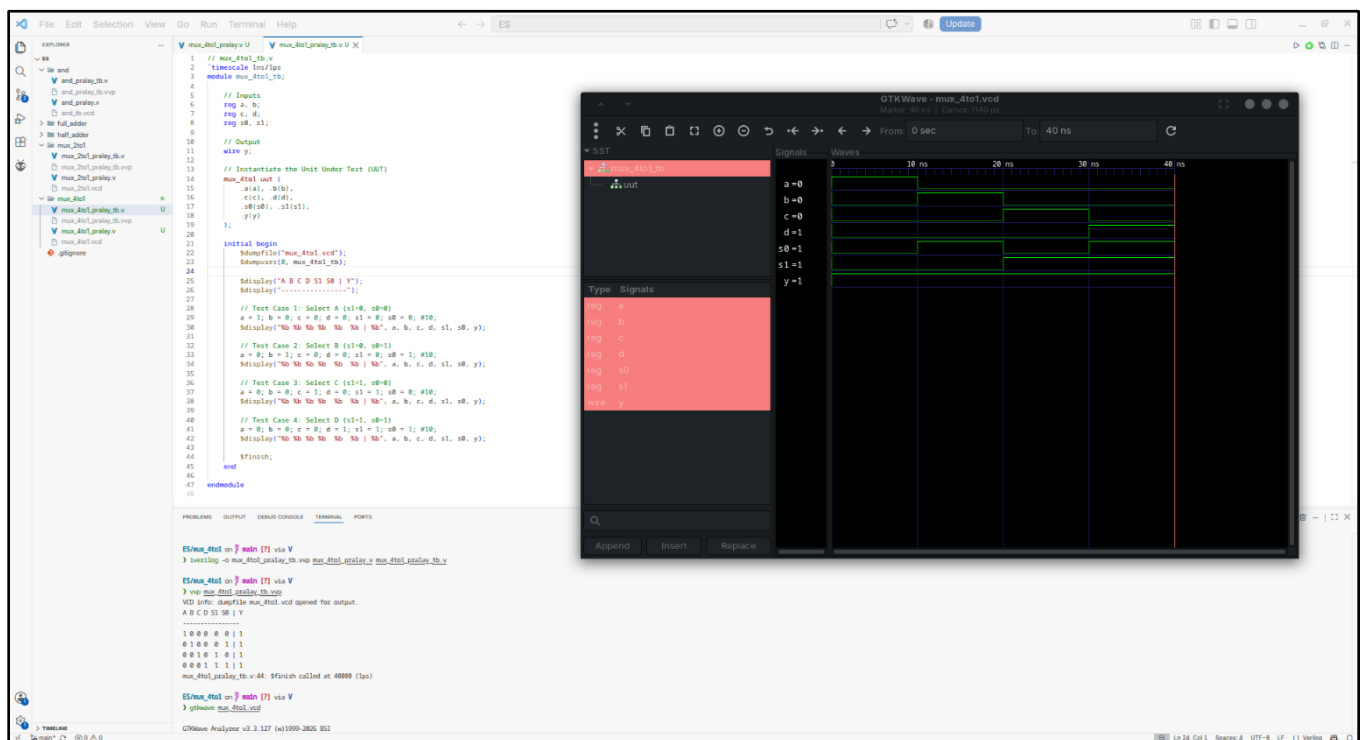
// Test Case 4: Select D (s1=1, s0=1)
a = 0; b = 0; c = 0; d = 1; s1 = 1; s0 = 1; #10;
$display("%b %b %b %b %b %b | %b", a, b, c, d, s1, s0,
y);

$finish;
end

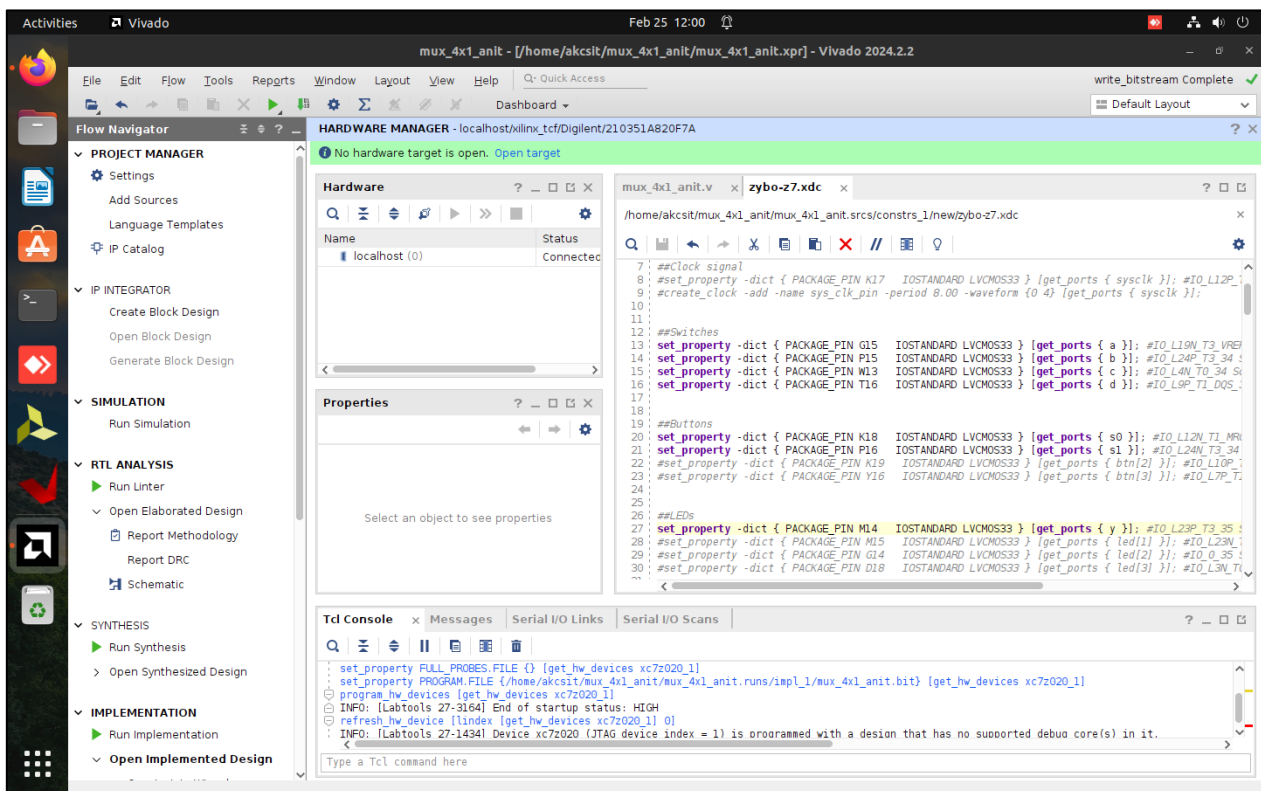
endmodule

```

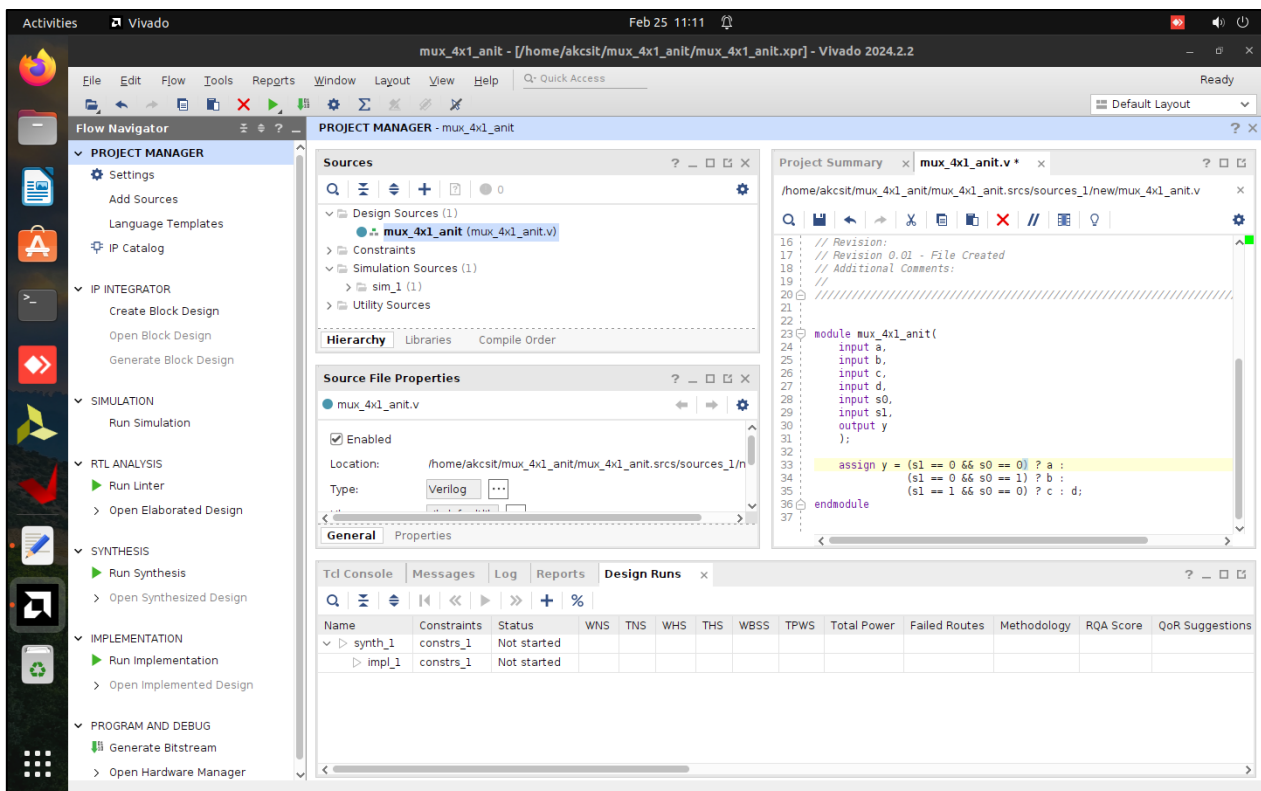
Output



Vivado Constrain File



Vivado Verilog File



Zybo-Z7-20 Output

